

Deterministic Execution Contracts for LLM Agent Systems: Architecture, Policy Enforcement, and Empirical Evaluation

Adam Tacon

AXIOM LLC, New Jersey

adam.tacon@proton.me axiom.co@proton.me

Abstract

Large language model (LLM) agents introduce a structural mismatch between probabilistic generation and the determinism required for safe, reproducible execution in consequential environments. We present AXIOM APEX (Autonomous Plan EXecutor), a plan-deterministic swarm runtime that enforces schema-validated, bounded, process-isolated execution contracts over LLM-generated plans, governed by a native pre-execution policy enforcement layer (the AXIOM APEX Validator) that validates and sanitizes plan submission prior to execution. Together, these systems instantiate a two-stage architecture in which generation remains probabilistic but execution is structurally constrained: given a validated plan, the tool-call sequence is deterministic; LLM variance across plan generation is logged and replayable. We formalize the information-flow discipline underlying this design, describe the implementation of key invariants including step bounding, tool timeout, blast-radius enforcement, and rollback generation, and report empirical results from a benchmark harness yielding a 12/12 pass rate with a composite `apex_score` of 1.008 across nominal and adversarial tool-invocation, file I/O, memory, and HTTP tasks. We further describe the Benchmark-Driven Self-Optimization (BDSO) loop integrated into APEX, bounded by a fitness function and restricted to a verified-eligible file set that permanently excludes security-critical components. The system is released as open-source infrastructure for deterministic AI agent deployment.

1. Introduction

The dominant failure mode of deployed LLM agent systems is not generative inaccuracy but architectural unsafety: the absence of enforceable boundaries on information flow between probabilistic generation and consequential execution. When an agent ingests tool outputs, retrieved documents, and user instructions through a unified untyped context, data and control become structurally indistinguishable. This ambiguity is the root cause of prompt injection, policy drift, unverified plan execution, and unstable self-modification.

Prior work has addressed these failure modes largely through heuristic filtering, constitutional prompting, and output classifiers — interventions that operate

at the semantic layer and inherit its fundamental unreliability. We argue that reliable agent execution requires a lower-level intervention: typed intermediate representations, schema-validated plan contracts, and execution environments with formally bounded resource consumption.

This paper makes the following contributions:

1. A formal characterization of information-flow collapse as the unifying cause across injection, verification failure, and unstable self-improvement.
2. APEX: a plan-deterministic agent runtime implementing a pure `run(task, config, registry) → State` execution contract with bounded step count, tool timeout, output limits, and JSONL audit trace.
3. A native Policy Validation Layer implementing blast-radius classification, tool allowlisting, and automatic rollback plan generation against the APEX execution substrate.
4. A benchmark harness and fitness function (`apex_score`) producing verifiable, reproducible evaluation of agent task execution.
5. A Benchmark-Driven Self-Optimization (BDSO) loop bounded by the fitness function and restricted to a security-invariant eligible file set.

2. Related Work

Prompt injection and control-flow integrity. Perez and Ribeiro (2022) characterize prompt injection as an input-validation failure; Greshake et al. (2023) extend this to indirect injection via retrieved content. Our treatment formalizes the mechanism as type confusion over a semantically untyped channel, analogous to memory-unsafe control-flow corruption.

Plan verification. SayCan (Ahn et al., 2022) and ToolFormer (Schick et al., 2023) generate tool-calling plans but perform no pre-execution structural validation. ReAct (Yao et al., 2022) interleaves reasoning and action without schema enforcement. APEX introduces a validation stage that rejects plans failing schema, step-count, or tool-allowlist constraints prior to any execution.

Agent safety frameworks. Constitutional AI (Bai et al., 2022) and RLHF-based approaches address value alignment at training time. Our approach is complementary and orthogonal: we address execution-time structural safety independently of model-level alignment.

Policy enforcement and execution contracts. Existing orchestration frameworks lack the pre-execution enforcement layer that distinguishes our native validator from prior work. LangGraph provides a stateful execution graph but no policy layer: tool invocation is unrestricted by blast-radius or allowlist constraints. Temporal enforces workflow durability and retry semantics but has no LLM-native contract mechanism — policies are defined in application code, not validated against plan structure. AutoGen supports multi-agent coordination but provides no execution bounds on individual agent

actions. Our native validator’s distinguishing contribution is enforcement at the API boundary: blast-radius classification, tool allowlisting, and step-count limits are validated against the plan structure before any execution begins.

Recursive self-improvement. Schmidhuber (2003) and subsequent work characterize RSI stability requirements. APEX instantiates a pragmatic bounded self-optimization loop (BDSO) with multi-candidate patch generation, isolated scoring, and a permanent security boundary excluding the plan auditor from eligible modification targets.

3. Theoretical Framework: Information Flow as the Core Safety Primitive

3.1 The Type-Confusion Model of Prompt Injection

LLM agents process all inputs — user prompts, tool outputs, retrieved documents — through a single untyped context. The model assigns no structural distinction between data (what is known) and control (what is done). A string that appears descriptive may be reinterpreted as prescriptive if it aligns with latent training patterns.

This is structurally equivalent to type confusion in memory-unsafe languages: a buffer interpreted as a control structure can overwrite execution flow; a string interpreted as an instruction can override policy constraints. The agent does not resolve this ambiguity through judgment — it lacks the structural capacity to prevent reinterpretation.

The critical consequence is **compositionality**: individual fragments may be benign, but their accumulation across retrieval, tool output, and memory can constitute a coherent adversarial program that emerges through interaction, never existing explicitly at any single point. Local per-input filtering is therefore provably insufficient. Safety must be defined over entire execution traces.

3.2 Verification as a Correctness Envelope

Verification in LLM agent systems cannot achieve full formal correctness — plans are underspecified, environments are non-deterministic, and verification latency must remain bounded for real-time operation. The pragmatic objective is a **correctness envelope**: a bounded region within which all executions are guaranteed to satisfy safety constraints.

Critically, this envelope must be **temporal and relational**: an action safe in isolation may violate constraints in combination with prior or subsequent steps. Verification must therefore operate over the complete plan structure, including state transitions and resource consumption projections, not over individual steps.

3.3 Recursive Self-Improvement and Stability Constraints

RSI dynamics are most accurately modeled as a feedback system. Stability requires that errors decay across iterations — each modification cycle must reduce, not amplify, deviation from desired behavior. Two conditions are necessary: (1) the magnitude of per-iteration changes must be bounded to keep verification tractable, and (2) the verification mechanism must scale with capability growth. A system that improves faster than its verifier can evaluate enters a regime of uncontrolled capability increase.

A further invariant is that **security-critical components must be excluded from the eligible modification set**. A self-modifying system that can alter its own safety enforcement is not self-improving — it is self-destabilizing.

4. APEX: Deterministic Agent Runtime

4.1 Core Execution Contract

APEX implements a pure function:

```
run(task: str, config: Config, registry: ToolRegistry) → State
```

The function is stateless with respect to prior runs. All state is local to the invocation and flushed to the audit log on completion. This purity is a primary architectural invariant: no hidden state accumulates across runs, and given a validated plan, execution is fully reproducible — identical plans produce identical tool-call sequences. LLM variance occurs at plan generation time and is captured in the audit log rather than eliminated.

4.2 Execution Bounds

Every run is subject to hard resource bounds:

Constraint	Bound
Maximum steps	32
Tool timeout	300 s (SIGALRM, main thread)
Output size	10 MB
Plan schema	Pydantic-validated pre-execution

Plans failing schema validation are rejected before any tool invocation. Step count is enforced as a hard ceiling, not a soft advisory. Tool timeout is enforced at the syscall boundary via `signal.alarm`, ensuring no tool invocation can hold the executor indefinitely.

4.3 Tool Registry and Autoloading

Tools are registered against a typed interface requiring an `effect(args) → result` method and a JSON schema for argument validation. The runtime autoloads tools from `~/.apex/tools/`, enabling deployment-specific tool sets without modifying core execution logic. This separation maintains the KISS invariant: the core executor has no awareness of domain-specific tool semantics.

4.4 Audit Infrastructure

Every run emits a JSONL trace recording each step's tool invocation, arguments, result, and wall time. Runs are persisted to a SQLite database (`~/.apex/runs.db`) with schema:

```
runs(id, task, plan_json, exit_code, token_count, wall_seconds, timestamp)
events(id, run_id, step, tool, args_json, result_json, timestamp)
```

This provides complete replay fidelity and supports post-hoc audit of any execution. The `apex replay` subcommand supports three modes: `simulate` (reconstruct trace without re-executing), `dry` (execute plan without committing side effects), and `live` (full re-execution).

4.5 Multi-Provider Abstraction

The LLM provider is abstracted behind a `providers.py` shim with concrete adapters for Gemini 2.5 Flash and Ollama. Provider selection is configuration-driven at startup; the core execution loop has no provider-specific logic. This isolation ensures that provider substitution does not require modification of any safety-critical component.

4.6 HTTP API

APEX exposes a Flask HTTP API (`apex serve`) with endpoints for run submission (POST `/run`), run history (GET `/runs`, GET `/runs/<id>`), replay (POST `/replay`), and export (GET `/export`). The server runs in single-threaded mode (`threaded=False`) to maintain SIGALRM compatibility — the tool timeout mechanism operates only on the main thread, and multi-threaded dispatch would silently disable it.

Authentication is enforced via `X-Apex-Key` header matched against `APEX_API_KEY` environment variable. If the variable is unset, the server starts without authentication with a startup warning.

5. Native Policy Enforcement Layer

5.1 Architecture

The AXIOM APEX Validator is a pre-execution policy enforcement layer that validates and governs plan submission to the executor. It implements a strict separation: the core loop defines execution semantics, while the validator defines submission policy. Plans that violate policy are rejected before reaching the execution loop.

5.2 Policy Schema

System policies are Pydantic-validated contracts:

```
class Policy(BaseModel):
    max_steps: int # default 16, bounds [1, 32]
    allowed_tools: list[str] # empty = unrestricted
    blast_radius: Literal["none", "local", "network"] # default "local"
    rollback_on_failure: bool # default True
```

The `blast_radius` field classifies the execution's potential impact envelope. `none` permits read-only operations; `local` permits filesystem modification within the process working directory; `network` permits egress. Policy is enforced at the structural level prior to execution, not as a runtime advisory.

5.3 Validation Logic

The validator enforces:

1. `len(plan.steps) > policy.max_steps`
2. No step invokes a tool in the blocked set (`{"shell"}` by default)
3. If `policy.allowed_tools` is non-empty, every step's tool must appear in it
4. No step invokes a tool in the `blast_radius`-derived block set (`blast_radius="none"` blocks `write_file`, `delete_file`, `http_get`, `http_post`; `blast_radius="local"` blocks `http_get`, `http_post`)

Violations produce structured rejection results with per-violation diagnostics. Validated plans are submitted to `POST /run` via the APEX HTTP API with the `X-Apex-Key` credential.

5.4 Rollback Generation

On execution failure, the system generates a compensating rollback plan by traversing the `events` table for the failed run in reverse step order and applying the reversal map:

`write_file` → `delete_file`

`shell` invocations are flagged to `stderr` as non-reversible. The rollback plan is itself subject to policy validation before submission. If no reversible steps exist, rollback returns `None` without error.

6. Benchmark-Driven Self-Optimization (BDSO)

6.1 Bounded BDSO Loop

APEX implements a Benchmark-Driven Self-Optimization loop (`apex rsi`) that generates candidate patches to the executor's own source files, scores them via the benchmark harness, and applies improvements that exceed a fitness threshold.

Each BDSO cycle generates $N=3$ candidate patches via the LLM, executes them in isolated subprocesses, scores each against the `apex_score` fitness function, and applies the highest-scoring candidate that exceeds the baseline. If no candidate improves on the baseline, the cycle terminates without modification. The mechanism is bounded and empirically grounded: improvement is defined by measurable benchmark regression, not open-ended capability growth.

6.2 Fitness Function

The `apex_score` composite fitness function is:

`apex_score = pass_rate × speed_factor × token_efficiency`

Where:

- `pass_rate = passed_tasks / total_tasks`
- `speed_factor = max(0.01, 1.0 - (avg_duration_seconds - 10.0) / 200.0)`
- `token_efficiency = max(0.01, 1.0 - (avg_tokens - 1000.0) / 50000.0)`

This formulation penalizes both execution latency and token overconsumption while treating task success as the primary signal. A system that passes all tasks at 10s/task with 1000 tokens/task achieves `apex_score = 1.0`.

6.3 Security Boundary

The BDSO eligible file set is restricted to `{loop.py, planner.py, llm.py}`. The plan auditor (`paranoid.py`) is **permanently excluded** from the eligible set. This exclusion is architectural, not configurable: a self-modifying system that can alter its own safety enforcement mechanism provides no meaningful safety guarantee.

7. Evaluation

7.1 Benchmark Harness

The benchmark harness (`apex.bench`) loads a task suite from `tasks.json`, executes each task via the APEX CLI in a subprocess, and records per-task pass/fail, duration, and exit code. Tasks specify an optional content check against stdout or a designated output file.

7.2 Task Suite

The benchmark suite comprises 12 tasks across nominal and adversarial categories:

Task ID	Category	Description	Tool(s)
<code>write_file</code>	Nominal	Write string to <code>/tmp</code> path, verify content	<code>write_file</code>
<code>shell_echo</code>	Nominal	Execute shell command, verify stdout	<code>shell</code>
<code>read_file</code>	Nominal	Write then read file, verify round-trip	<code>read_file</code>
<code>memory_write_read</code>	Nominal	Write key-value to memory, retrieve and verify	<code>memory</code>
<code>http_get</code>	Nominal	HTTP GET to public endpoint, verify response	<code>http_get</code>
<code>multi_step_chain</code>	Adversarial	Multi-step chained file write and read	<code>write_file</code> , <code>read_f</code>
<code>overwrite_existing</code>	Adversarial	Write to pre-existing path, verify overwrite	<code>write_file</code>
<code>shell_pipeline</code>	Adversarial	Execute shell pipeline, verify stdout	<code>shell</code>
<code>http_post</code>	Adversarial	HTTP POST to endpoint, verify response	<code>http_post</code>
<code>memory_multi_key</code>	Adversarial	Write multiple keys, retrieve and verify all	<code>memory</code>
<code>large_file_write</code>	Adversarial	Write large payload, verify size and content	<code>write_file</code>
<code>multi_file_verify</code>	Adversarial	Write multiple files, read and verify each	<code>write_file</code> , <code>read_f</code>

The adversarial suite targets boundary conditions identified as coverage gaps in prior evaluation: multi-step plan chaining, idempotency under overwrite, I/O scaling, and multi-key memory operations. Runtime blast-radius isolation (chroot/namespace enforcement) remains a near-term objective; current `blast_radius` enforcement operates at plan-validation time.

7.3 Results

Two independent benchmark runs were executed on hardware (AMD Ryzen 3 3250U, 33 GiB RAM, NVMe storage) under identical conditions with a 180-second per-task timeout:

Run	Pass Rate	Wall Time	<code>speed_factor</code>	<code>token_efficiency</code>	<code>apex_score</code>
Run 1	12/12 (1.0)	92.0 s	1.0117	0.9959	1.007552
Run 2	12/12 (1.0)	90.5 s	1.0123	0.9954	1.007643

`apex_score > 1.0` in both runs indicates execution faster than the 10s/task baseline with near-unity token efficiency. Results are consistent across runs (< 0.0001). All 12 tasks completed within timeout; the `memory_multi_key` task exhibits the longest per-task wall time (~22–25 s) due to multi-round memory retrieval, accounted for by the 180 s timeout budget.

8. Docker Deployment

APEX is containerized and orchestrated via docker-compose. The compose stack defines the primary execution container, which builds directly from the source, binds port 8080, and exposes an automated healthcheck on GET /health while persisting configuration states, database records, and run logs to a named volume. This allows production deployment of the full plan-deterministic architecture as a single docker-compose up command with credentials securely injected at runtime.

9. Future Work

Blast-radius runtime enforcement. `blast_radius` is currently enforced at plan-validation time. Full runtime enforcement requires process isolation (chroot or namespace) for `local` and an egress proxy for `network`. Plan-level enforcement is a necessary but not sufficient condition for the security guarantee the field implies.

Multi-agent consensus. For high-`blast_radius` tasks, generating $N=3$ independent plans and requiring intersection before execution would substantially reduce the probability of unilateral consequential actions.

RAG-driven policy grounding. Policy constraints derived from compliance manifests (SOC2, HIPAA) via retrieval-augmented generation would enable domain-specific policy without manual specification.

BDSO fitness expansion. The current `apex_score` formulation does not include a security-regression component. Adding a paranoid-flag delta term would penalize BDSO cycles that reduce plan auditor sensitivity.

10. Conclusion

The reliable deployment of LLM agents in consequential environments requires architectural discipline that probabilistic generation alone cannot provide. The central invariant is information-flow separation: generation proposes, a typed intermediate representation constrains, and a bounded executor acts. AXIOM

APEX instantiates this architecture concretely, combining a deterministic execution loop with a native pre-execution policy validator, complete audit infrastructure, and a BDSO loop bounded by a composite fitness function and a permanent security exclusion boundary.

The empirical results — 12/12 pass rate across nominal and adversarial tasks, `apex_score` ~1.008 across two independent runs — establish a reproducible baseline against which future capability and hardening improvements can be measured. The architecture is released as open-source infrastructure at github.com/axiom-llc/axiom-apex.

References

- Ahn, M., et al. (2022). Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. *arXiv:2204.01691*.
- Bai, Y., et al. (2022). Constitutional AI: Harmlessness from AI Feedback. *arXiv:2212.08073*.
- Greshake, K., et al. (2023). Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. *arXiv:2302.12173*.
- Perez, F., & Ribeiro, I. (2022). Ignore Previous Prompt: Attack Techniques For Language Models. *arXiv:2211.09527*.
- Schick, T., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv:2302.04761*.
- Schmidhuber, J. (2003). Gödel Machines: Self-Referential Universal Problem Solvers Making Provably Optimal Self-Improvements. *Technical Report IDSIA-19-03*.
- Yao, S., et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv:2210.03629*.

Appendix A: APEX Invariants

The following invariants are maintained across all versions of the system:

1. `run()` is a pure function — no hidden state persists across invocations
2. No async primitives or threading in the core execution path
3. Plan-deterministic outputs: given a validated plan, identical inputs produce identical tool-call sequences; LLM variance at plan generation time is logged and replayable
4. Bounded execution: `max_steps=32`, `timeout=300s`, `output=10MB`
5. All plans are schema-validated before any tool invocation

6. Pre-execution policy enforcement is always active
7. `paranoid.py` is not BDSO-eligible — permanent security boundary
8. Flask HTTP API runs `threaded=False` to preserve SIGALRM semantics

Appendix B: Benchmark Task Suite (tasks.json, nominal subset)

```
[
  {
    "id": "write_file",
    "prompt": "write 'hello world' to /tmp/apex-bench-out.txt",
    "check": "hello",
    "check_file": "/tmp/apex-bench-out.txt"
  },
  {
    "id": "shell_echo",
    "prompt": "run the shell command: echo apex-bench-ok",
    "check": "apex-bench-ok"
  },
  {
    "id": "read_file",
    "prompt": "write 'readtest' to /tmp/apex-bench-read.txt then read it back and output the",
    "check": "readtest"
  },
  {
    "id": "memory_write_read",
    "prompt": "store the value 'bench-value' under key 'bench-key' in memory, then retrieve",
    "check": "bench-value"
  },
  {
    "id": "http_get",
    "prompt": "make an HTTP GET request to https://httpbin.org/get and output the response",
    "check": "httpbin"
  }
]
```

The full 12-task suite including adversarial cases is maintained at `apex/benchmarks/tasks.json` in the APEX repository.